NAZARBAYEV UNIVERSITY

Programming for Engineers

Week 6: Recursion and Dictionaries

Yanwei Wang
School of Engineering
Nazarbayev University

Fall 2026



Week 6 at a glance

Lecture 16

Recursion versus iteration
Base cases, recursive cases, tracing, call stacks, factorial, and recursive bisection.

Lecture 17

Richer recursive structures
Multiple base cases, several recursive calls, strings, lists, and non-numerical problems.

Lecture 18

Dictionaries
Key-value data, safe lookup, nested records, engineering metadata, and Assignment 3.

Reduce the problem → Reach a base case → Rebuild the result → Store meaning by key

NAZARBAYEV UNIVERSITY

LECTURE 16

Recursion versus Iteration

Solving a problem by reducing it to a smaller version of itself

NAZARBAYEV UNIVERSITY

Lecture 16 learning goals

By the end of this lecture, you should be able to:

1. explain what makes a function recursive;
2. identify a base case and a recursive case;
3. determine whether each call moves toward termination;
4. trace recursive calls and returned values;
5. explain the role of the call stack;
6. implement factorial recursively and iteratively;
7. compare recursion with a loop;
8. recognize missing-base-case and wrong-direction errors;
9. test recursive functions with small and boundary cases;
10. express the Week 3 bisection idea recursively.

A recursive call must receive a smaller or simpler problem

NAZARBAYEV UNIVERSITY

Warm-up: what repeats, and what becomes smaller?

Three familiar processes

1. Count down from 5 to 0.
2. Compute $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$.
3. Bisect an interval until it is sufficiently narrow.

Discuss

For each process, identify the stopping condition and the quantity that becomes smaller at every step.

Pairs - 3 min

What is recursion?

Definition

A recursive function is a function that calls itself, directly or indirectly, to solve a smaller or simpler version of the same problem.

Base case

A case that can be answered immediately, without another recursive call.

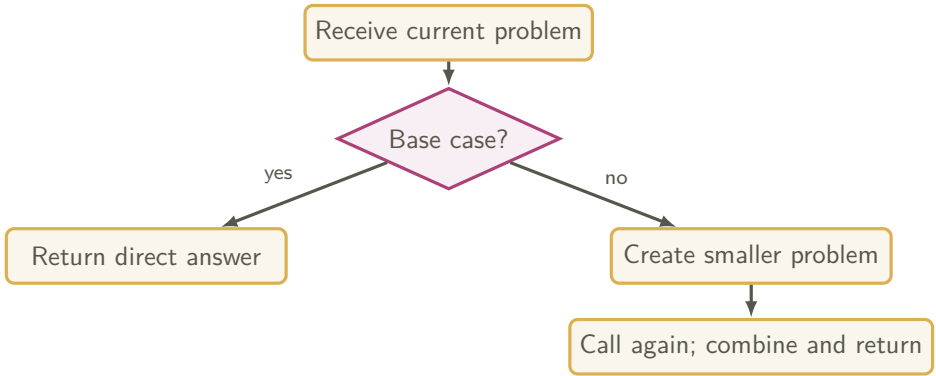
Recursive case

A rule that reduces the current input and combines the smaller result into the current answer.

Stop directly, or reduce and call again.

NAZARBAYEV UNIVERSITY

The two-part recursive pattern



NAZARBAYEV UNIVERSITY

A first recursive function: countdown

```
def countdown(n):
    if n <= 0:
        print("Launch")
        return

    print(n)
    countdown(n - 1)

countdown(3)
```

Output

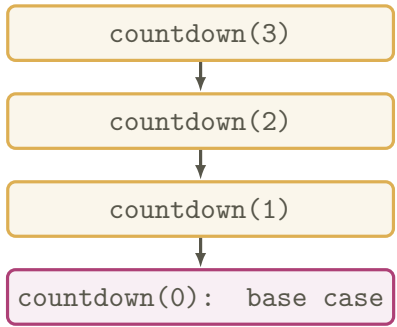
```
3
2
1
Launch
```

Progress

Each call reduces `n` by 1.

NAZARBAYEV UNIVERSITY

Trace the calls before running the code



Trace question

Which line prints before the next call? What changes if `print(n)` is moved below the recursive call?

NAZARBAYEV UNIVERSITY

Prediction: code before or after the recursive call

Version A

```
def show_a(n):
    if n == 0:
        return
    print(n)
    show_a(n - 1)
```

Version B

```
def show_b(n):
    if n == 0:
        return
    show_b(n - 1)
    print(n)
```

Predict

What do `show_a(3)` and `show_b(3)` print? Explain the order without executing either function.

Individual - 3 min

NAZARBAYEV UNIVERSITY

The base case is a contract, not decoration

A working base case

Covers the smallest valid input and returns without another recursive call.

Questions to ask

Is the base case reachable? Does it return the correct neutral or terminal result? Are all valid paths covered?

Termination argument

State a quantity that is nonnegative and becomes strictly smaller with every recursive call until the base case is reached.

NAZARBAYEV UNIVERSITY

What happens without a working base case?

```
def broken_countdown(n):
    print(n)
    broken_countdown(n - 1)

broken_countdown(3)
```

Eventually

```
RecursionError: maximum recursion
  depth exceeded
```

Meaning

Python stopped the growing chain of calls. It did not prove the algorithm correct.

A subtler failure: moving away from the base case

```
def countdown(n):
    if n == 0:
        return
    print(n)
    countdown(n + 1)
```

Diagnosis

A base case exists, but calls starting from a positive `n` move farther from zero.

Repair

Change the recursive argument so a measurable distance to the base case decreases.

A base case is useful only when the recursion approaches it

NAZARBAYEV UNIVERSITY

Factorial has a natural recursive definition

For a nonnegative integer n,

$$0! = 1, \qquad n! = n(n-1)! \quad \text{for } n > 0.$$

Base case

$0! = 1$ is known directly.

Recursive case

To calculate $n!$, calculate the smaller problem $(n-1)!$ and multiply by n.

Recursive factorial in Python

```
def factorial_recursive(n):
    """Return n! for a nonnegative integer n."""
    if n == 0:
        return 1
    return n * factorial_recursive(n - 1)

print(factorial_recursive(5))
```

Output

```
120
```

Smaller input

`n - 1` eventually reaches 0.

NAZARBAYEV UNIVERSITY

Tracing factorial(4): calls expand

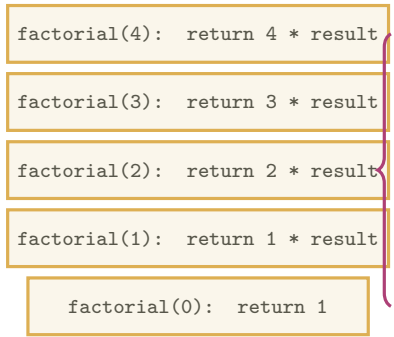
$$
\begin{aligned}
\text{factorial}(4) &= 4\,\text{factorial}(3) \\
&= 4 \cdot 3\,\text{factorial}(2) \\
&= 4 \cdot 3 \cdot 2\,\text{factorial}(1) \\
&= 4 \cdot 3 \cdot 2 \cdot 1\,\text{factorial}(0) \\
&= 4 \cdot 3 \cdot 2 \cdot 1 \cdot 1.
\end{aligned}
$$

Do not skip the base case

The multiplication cannot finish until `factorial(0)` returns 1.

NAZARBAYEV UNIVERSITY

The call stack remembers unfinished work

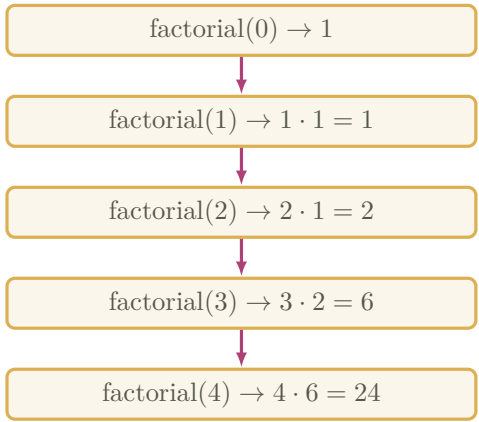


Stack behavior

The most recent call finishes first. Each waiting call keeps its local value of `n` and the operation still to perform.

NAZARBAYEV UNIVERSITY

Unwinding returns the result



NAZARBAYEV UNIVERSITY

The same factorial calculation with iteration

```
def factorial_iterative(n):
    """Return n! for a nonnegative integer n."""
    result = 1
    for value in range(1, n + 1):
        result *= value
    return result

print(factorial_iterative(5))
```

State update

One function call repeatedly updates `result`.

Output

```
120
```

NAZARBAYEV UNIVERSITY

Recursion and iteration express different mental models

Question	Iteration	Recursion
What repeats?	A loop body	A function call
What stores progress?	Variables updated in one frame	A stack of active call frames
How does it stop?	Loop condition becomes false	A base case returns directly
How is progress shown?	Counter/state approaches a limit	Recursive input becomes smaller
Typical beginner risk	Infinite loop	Missing/unreachable base case
Often best for	Repeated engineering calculations	Naturally self-similar or nested problems

NAZARBAYEV UNIVERSITY

Which should an engineer choose?

Prefer a loop when

The process is naturally sequential, performance and memory matter, or a state variable is updated repeatedly.

Consider recursion when

The definition is recursive, the data are nested, or each step creates one or more smaller instances of the same problem.

Engineering judgment

Correctness comes first. Choose the form that makes the stopping argument, assumptions, and data structure easiest to verify.

NAZARBAYEV UNIVERSITY

Validate the recursive function's input contract

```
def factorial_recursive(n):
    """Return n! for a nonnegative integer n."""
    if not isinstance(n, int) or n < 0:
        return None
    if n == 0:
        return 1
    return n * factorial_recursive(n - 1)
```

Why validate?

A negative or non-integer input does not satisfy this definition and may never reach the intended base case.

Week 7 preview

Later we will handle invalid inputs more explicitly with exceptions and assertions.

NAZARBAYEV UNIVERSITY

Test cases for recursive factorial

Test class	Input and expectation	What it checks
Base case	$n = 0 \to 1$	direct termination and neutral value
Near-base case	$n = 1 \to 1$	one recursive step
Known value	$n = 5 \to 120$	multiplication and unwinding
Invalid boundary	$n = -1 \to$ `None`	input contract
Invalid type	$n = 2.5 \to$ `None`	type assumption
Agreement test	recursive $=$ iterative	two implementations agree for several small n

Testing principle

Test the base case first. If the stopping case is wrong, every larger result is at risk.

NAZARBAYEV UNIVERSITY

Tracing activity: fill the call table

```
def product_to_one(n):
    if n <= 1:
        return 1
    return n * product_to_one(n - 1)

answer = product_to_one(4)
```

Before running

List each call, identify the base case, show the returned value from every frame, and predict `answer`.

Pairs - 5 min

NAZARBAYEV UNIVERSITY

Debugging clinic: three recursion defects

A: no base

```
def f(n):
    return n * f(n-1)
```

What stops the calls?

B: wrong direction

```
def f(n):
    if n == 0:
        return 1
    return n * f(n+1)
```

Does the input shrink?

C: missing return

```
def f(n):
    if n == 0:
        return 1
    n * f(n-1)
```

What value is returned?

Repair method

Write the base case, write the smaller input, then state exactly how the smaller answer becomes the current answer.

Recursion depth is a resource

Every active call stores information

Parameters, local variables, and the return location occupy a call-stack frame until the call finishes.

A practical limit exists

Very deep recursion can exhaust the permitted call depth even when the mathematical algorithm would eventually terminate.

Engineering implication

For a long time series or millions of repeated steps, a loop is usually clearer and less memory-intensive than one recursive call per value.

NAZARBAYEV UNIVERSITY

Week 3 revisited: bisection is self-similar

To solve $f(x) = 0$ on $[a, b]$:

1. compute the midpoint $m = (a + b)/2$;
2. if the residual or interval is sufficiently small, return m;
3. otherwise retain the half-interval that still brackets a root;
4. solve the same problem on that smaller interval.

Recursive viewpoint

The next problem has the same structure as the current problem, but its interval is half as wide.

NAZARBAYEV UNIVERSITY

A recursive bisection function

```
def bisection_recursive(function, left, right, tolerance):
    midpoint = (left + right) / 2

    if abs(function(midpoint)) < tolerance:
        return midpoint

    if function(left) * function(midpoint) < 0:
        return bisection_recursive(
            function, left, midpoint, tolerance)
    return bisection_recursive(
        function, midpoint, right, tolerance)
```

Assumption

The starting interval must be suitable: the function is continuous and the endpoints bracket a sign change. A robust version checks this explicitly.

NAZARBAYEV UNIVERSITY

Use recursive bisection on an engineering target

```
def residual_time(t_s):
    # Height model: h(t) = 20 - 0.5*g*t^2
    g = 9.81
    return 20.0 - 0.5 * g * t_s**2

impact_time_s = bisection_recursive(
    residual_time, left=0.0, right=3.0,
    tolerance=1e-6)
print(f"Impact time = {impact_time_s:.4f} s")
```

Interpretation

The function finds a time at which the modeled height is approximately zero.

Model limit

The calculation neglects air resistance and assumes constant gravitational acceleration.

NAZARBAYEV UNIVERSITY

A recursion design checklist

1. State the valid input domain and expected output.
2. Identify every base case before writing the recursive case.
3. Define a measurable problem size or distance to termination.
4. Ensure every recursive call makes that measure smaller.
5. Explain how returned sub-results combine into the current result.
6. Trace a tiny example by hand, including the return path.
7. Test base, near-base, normal, and invalid cases.
8. Compare with an iterative formulation when clarity or efficiency is uncertain.

NAZARBAYEV UNIVERSITY

Lecture 16 exit ticket

For the function below, answer without running it:

$$R(n) = \begin{cases} 0, & n = 0, \\ n + R(n-1), & n > 0. \end{cases}$$

1. What is the base case?
2. What gets smaller?
3. Expand $R(3)$ until the base case.
4. What does $R(n)$ calculate?
5. Would you implement this with recursion or iteration in a large engineering program, and why?

Submit before leaving

NAZARBAYEV UNIVERSITY

LECTURE 17

Multiple Base Cases and Non-Numerical Recursion

Recursive definitions can branch, process strings, and traverse structured data

NAZARBAYEV UNIVERSITY

Lecture 17 learning goals

By the end of this lecture, you should be able to:

1. explain why some problems require several base cases;
2. trace a function containing more than one recursive call;
3. implement the Fibonacci recurrence;
4. recognize repeated work in a recursion tree;
5. compare recursive and iterative sequence generation;
6. apply recursion to strings and lists;
7. design a recursive palindrome test;
8. identify the smaller non-numerical input;
9. use short-circuit logic safely in recursion;
10. repair recursive functions that fail to shrink their data.

The problem size can be a number, a string length, or a collection length

NAZARBAYEV UNIVERSITY

Warm-up: a sequence needs two starting values

Consider the sequence

$$1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13, \ldots$$

Every new value is the sum of the previous two.

Question 1

Why is one starting value not enough to determine the sequence?

Pairs - 3 min

Question 2

How many smaller subproblems are required to calculate the next value recursively?

NAZARBAYEV UNIVERSITY

Some recursive definitions need several base cases

General principle

Use as many base cases as are needed to cover all smallest valid inputs from which the recursive rule can proceed.

Fibonacci-style definition

$$F(0) = 1, \qquad F(1) = 1, \qquad F(n) = F(n-1) + F(n-2) \quad (n \geq 2).$$

Two differences from factorial

There are two base cases, and the recursive case makes two recursive calls.

Recursive Fibonacci implementation

```
def fibonacci(n):
    """Return F(n) for an integer n >= 0."""
    if n == 0 or n == 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(5))
```

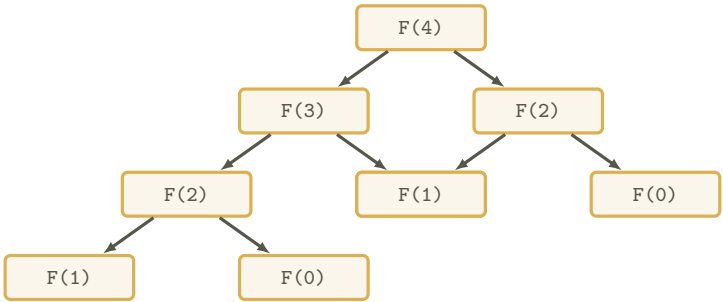
Output

```
8
```

Progress

Both recursive inputs are smaller than `n`.

Trace $\mathrm{F}(4)$ as a recursion tree



Observe

The same subproblem, such as `F(2)` or `F(1)`, may be calculated more than once.

NAZARBAYEV UNIVERSITY

The return values combine upward

$$F(0) = 1, \qquad F(1) = 1,$$
$$F(2) = F(1) + F(0) = 2,$$
$$F(3) = F(2) + F(1) = 3,$$
$$F(4) = F(3) + F(2) = 5,$$
$$F(5) = F(4) + F(3) = 8.$$

Combination rule

Each call returns the sum of two smaller answers. The caller waits until both are available.

NAZARBAYEV UNIVERSITY

Correct does not automatically mean efficient

What the recursive code does well

It mirrors the mathematical recurrence and clearly displays the two base cases.

What it does poorly

It recomputes the same values many times as `n` grows, creating a rapidly expanding call tree.

Course connection

Formal algorithmic efficiency is studied later. For now, notice repeated work and compare alternative formulations.

An iterative Fibonacci alternative

```
def fibonacci_iterative(n):
    if n == 0 or n == 1:
        return 1

    previous = 1
    current = 1
    for index in range(2, n + 1):
        previous, current = current, previous +
    current
    return current
```

State representation

Only the two most recent sequence values are retained.

Same result

`fibonacci_iterative(5)` returns 8.

NAZARBAYEV UNIVERSITY

Recursive versus iterative sequence generation

Feature	Recursive Fibonacci	Iterative Fibonacci
Matches recurrence	very directly	less directly
Active calls/state	many call frames	a few variables
Repeated subproblems	yes, in the naive version	no
Ease of hand tracing	good for small n	good with a state table
Practical choice here	useful for learning recursion	usually preferable for computation

NAZARBAYEV UNIVERSITY

Prediction activity: how many base cases?

```
def sequence(n):
    if n == 0:
        return 2
    if n == 1:
        return 3
    return 2 * sequence(n - 1) - sequence(n - 2)
```

Before running

Calculate `sequence(2)`, `sequence(3)`, and `sequence(4)` by tracing. Why are both base cases required?

Pairs - 5 min

Recursion is not limited to numbers

The recursive input only needs a measurable notion of "smaller."

String

Remove the first and last characters; length decreases.

List

Remove or skip the first element; number of elements decreases.

Nested structure

Process one node, then recursively process its smaller children or subcollections.

Palindrome as a recursive question

A string is a palindrome when it reads the same forward and backward.

Recursive idea

A string is a palindrome if:

1. it has length 0 or 1; or
2. its first and last characters match, and the interior substring is a palindrome.

Smaller problem

For a string `s`, the slice `s[1:-1]` removes two characters.

Recursive palindrome test

```
def is_palindrome(text):
    if len(text) <= 1:
        return True

    return (text[0] == text[-1]
            and is_palindrome(text[1:-1]))
```

Base cases

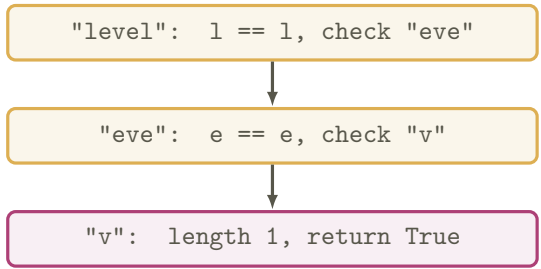
The empty string and every one-character string are palindromes.

Recursive case

Compare the boundary characters, then inspect the shorter interior.

NAZARBAYEV UNIVERSITY

Trace `is_palindrome("level")`



Unwinding

`"v"` returns True, then `"eve"` returns True, then `"level"` returns True.

NAZARBAYEV UNIVERSITY

Short-circuit evaluation can prevent unnecessary calls

In

```
text[0] == text[-1] and is_palindrome(text[1:-1]),
```

Python evaluates from left to right.

If boundary characters match

The recursive interior test is required.

If they do not match

The whole expression is already False, so the recursive call is skipped.

Trace a non-palindrome

```
print(is_palindrome("sensor"))
```

First comparison

`"s" != "r"`

Result

The function returns False immediately. It never tests `"enso"` because the first condition is already false.

NAZARBAYEV UNIVERSITY

Normalize human-entered text before recursion

```
def letters_only_lower(text):
    cleaned = ""
    for character in text.lower():
        if character.isalpha():
            cleaned += character
    return cleaned

def is_phrase_palindrome(text):
    cleaned = letters_only_lower(text)
    return is_palindrome(cleaned)
```

Separation of responsibilities

One function cleans the text; the recursive function answers the structural question. Document what characters and languages the cleaning rule supports.

NAZARBAYEV UNIVERSITY

Prediction: punctuation and capitalization

```
phrase = "Never odd or even"
print(is_phrase_palindrome(phrase))
```

Before running

Write the cleaned string. Then trace only the first three recursive levels. What is the expected final result?

Individual - 3 min

A recursive list pattern

To process a list recursively, separate it into:

$$\text{first element} + \text{remaining list}.$$

Base case

The empty list has no elements left to process.

Recursive case

Use `values[0]` now and recursively process `values[1:]`.

Recursive sum of a list

```
def recursive_sum(values):
    if len(values) == 0:
        return 0
    return values[0] + recursive_sum(values[1:])

readings = [2.0, 3.5, 4.0]
print(recursive_sum(readings))
```

Output

```
9.5
```

Neutral value

The sum of an empty list is 0.

Trace the list reduction

$$\begin{aligned}\text{sum}([2.0, 3.5, 4.0]) &= 2.0 + \text{sum}([3.5, 4.0]) \\ &= 2.0 + 3.5 + \text{sum}([4.0]) \\ &= 2.0 + 3.5 + 4.0 + \text{sum}([]) \\ &= 2.0 + 3.5 + 4.0 + 0 = 9.5.\end{aligned}$$

Data question

What should the function do if the list contains a missing-value code such as −999? Recursion does not remove the need for data validation.

Recursive search in a sequence

```
def contains_recursive(values, target):
    if len(values) == 0:
        return False
    if values[0] == target:
        return True
    return contains_recursive(values[1:], target)
```

Base case 1

No elements remain: target is absent.

Base case 2

The first element matches: target is present.

Why two base cases improve the search

Empty-list base case

Defines the result when the search space has been exhausted.

Match base case

Stops immediately when enough evidence has been found.

Connection to engineering monitoring

A scan can stop at the first threshold violation rather than processing every later reading.

NAZARBAYEV UNIVERSITY

A non-numerical recursion bug: the string never shrinks

```
def is_palindrome(text):
    if len(text) <= 1:
        return True
    return (text[0] == text[-1]
            and is_palindrome(text))
```

Diagnosis

The recursive call receives the same string. Its length never decreases.

Repair

Call the function with `text[1:-1]`.

Another bug: the base case is too narrow

```
def is_palindrome(text):
    if len(text) == 1:
        return True
    return (text[0] == text[-1]
            and is_palindrome(text[1:-1]))
```

Even-length input

A palindrome such as `"abba"` eventually produces the empty string, not a one-character string.

Repair

Use `len(text) <= 1` to cover both terminal forms.

NAZARBAYEV UNIVERSITY

A general recipe for non-numerical recursion

1. Define the structure: string, list, nested record, or tree-like object.
2. Choose a size measure, such as length or number of unprocessed elements.
3. Identify every smallest structure that can be answered directly.
4. Select one part to process now.
5. Create one or more strictly smaller structures.
6. Combine the current evidence with returned sub-results.
7. Test empty, singleton, short normal, and malformed inputs.

NAZARBAYEV UNIVERSITY

Design activity: recursively reverse a string

```
def reverse_recursive(text):
    # Complete this function.
    ...
```

Your design must state

The base case, the smaller substring, the combination rule, and expected outputs for `""`, `"A"`, and `"NU"`.

Pairs - 5 min

One possible recursive reversal

```
def reverse_recursive(text):
    if len(text) <= 1:
        return text
    return reverse_recursive(text[1:]) + text[0]

print(reverse_recursive("NU"))
```

Output

```
UN
```

Combination

Reverse the remaining substring, then append the first character.

NAZARBAYEV UNIVERSITY

Recursion review: four questions to answer aloud

For every recursive function, be able to state:

1. **What are the base cases?**
2. **What is smaller in each recursive call?**
3. **Why must the calls eventually stop?**
4. **How are the returned sub-results combined?**

Evidence standard

Saying "it works because it calls itself" is not an explanation. A correct explanation includes termination and the meaning of the returned value.

Lecture 17 exit ticket

Consider a recursive function that counts occurrences of a letter in a string.

1. What should the function return for an empty string?
2. How can the first character be processed?
3. What strictly smaller string should be passed to the next call?
4. Write the return expression in plain language.
5. Give one boundary test and one normal test.

Submit before leaving

NAZARBAYEV UNIVERSITY

LECTURE 18

Dictionaries and Assignment 3

Representing engineering information as named key-value relationships

NAZARBAYEV UNIVERSITY

Lecture 18 learning goals

By the end of this lecture, you should be able to:

1. explain the key-value model of a dictionary;
2. create and inspect dictionaries;
3. retrieve values using keys;
4. add, update, and remove entries;
5. check key membership before access;
6. use `get()`, `keys()`, `values()`, and `items()`;
7. iterate through named engineering data;
8. construct nested dictionaries and lists of dictionaries;
9. represent material properties and sensor metadata;
10. diagnose missing-key, inconsistent-key, and unit errors.

Use a dictionary when names or labels are more meaningful than positions

NAZARBAYEV UNIVERSITY

Warm-up: what does position 2 mean?

An ambiguous list record

```
sensor = ["T-101", "C", 80.0, True]
```

Discuss

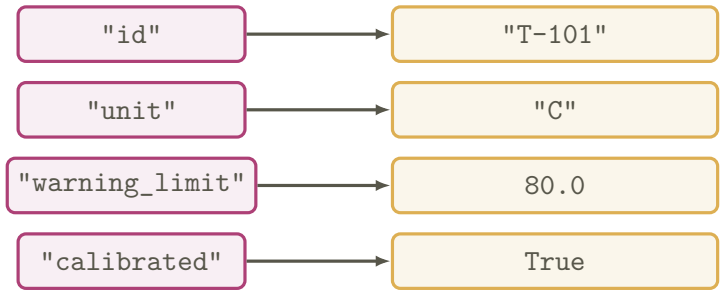
What does each position mean? How would a teammate know? What happens if a fifth property is inserted?

Pairs - 3 min

A better question

Can the program store each value together with a name that explains its meaning?

A dictionary is a mapping



Meaning

A key identifies a value. Access is based on the key, not on a numerical position.

Create a first dictionary

```
sensor = {
    "id": "T-101",
    "unit": "C",
    "warning_limit": 80.0,
    "calibrated": True
}
print(sensor)
print(type(sensor))
```

Syntax

Curly braces contain comma-separated `key: value` pairs.

Type

`dict`

NAZARBAYEV UNIVERSITY

Retrieve a value by key

```
sensor_id = sensor["id"]
limit_c = sensor["warning_limit"]
print(sensor_id)
print(limit_c)
```

Output

```
T-101
80.0
```

Read aloud

"Get the value associated with the key `warning_limit`."

NAZARBAYEV UNIVERSITY

What can be a key or a value?

Keys

Common beginner choices are strings, integers, and tuples of immutable values. Keys must be unique within one dictionary.

Values

Numbers, strings, Booleans, lists, tuples, dictionaries, functions, and many other Python objects.

Meaning and consistency

Prefer one clear key convention. For example, do not mix `"yield_strength"`, `"YieldStrength"`, and `"strength"` for the same property.

Add or update dictionary entries

```
material_density = {
    "steel": 7850,
    "aluminum": 2700
}
material_density["copper"] = 8960    # add
material_density["aluminum"] = 2710 # update
print(material_density)
```

One assignment pattern

A new key adds an entry; an existing key replaces its value.

Mutability

Preserve authoritative data or make an explicit copy before what-if modification.

A missing key raises `KeyError`

```
material_density = {
    "steel": 7850,
    "aluminum": 2700
}
print(material_density["titanium"])
```

Typical message

```
KeyError: 'titanium'
```

Interpretation

The requested key is absent. This may reflect a spelling mismatch, unrecognized input, or incomplete data.

NAZARBAYEV UNIVERSITY

Check membership before direct access

```
selected_material = "titanium"

if selected_material in material_density:
    density = material_density[selected_material]
    print(f"Density = {density} kg/m^3")
else:
    print("Material not found in database")
```

The `in` test checks keys

For a dictionary `d`, `key in d` asks whether the key exists, not whether a value appears in the dictionary.

Use `get()` when a default is meaningful

```
density = material_density.get("titanium", None)

if density is None:
    print("Material not found")
else:
    print(f"Density = {density} kg/m^3")
```

Pattern

`d.get(key, default)` returns the stored value or the chosen default.

Do not invent evidence

A default density of zero would be physically misleading. Use a sentinel such as `None` when absence must remain visible.

NAZARBAYEV UNIVERSITY

Keys, values, items, and dictionary iteration

```
print(material_density.keys())
print(material_density.values())
print(material_density.items())

for material in material_density:
    density = material_density[material]
    print(material, density)
```

Choose the view

Keys identify names, values support value-only processing, and items provide both.

Default iteration

`for key in dictionary` iterates over keys.

NAZARBAYEV UNIVERSITY

Loop over key-value pairs with `items()`

```
print("MATERIAL DENSITY TABLE")
print("----------------------")
for material, density in material_density.items():
    print(f"{material:10s}: {density:5d} kg/m^3")
```

Why `items()` is useful

The loop unpacks each pair into meaningful names, avoiding a second lookup expression.

Remove entries deliberately

Delete by key

```
del material_density["copper"]
```

Use when the key is known to exist.

Remove and retrieve

```
removed = material_density.pop(
    "copper", None)
```

A default avoids a missing-key error.

Data governance

Do not silently delete measurements or reference properties. Document why an entry was removed and preserve the source when required.

NAZARBAYEV UNIVERSITY

Engineering lookup table: material density

```
material_density_kg_m3 = {
    "steel": 7850,
    "aluminum": 2700,
    "copper": 8960
}

selected = input("Material: ").strip().lower()
density = material_density_kg_m3.get(selected)
if density is None:
    print("Unknown material")
else:
    print(f"Density = {density} kg/m^3")
```

Good habits

Unit in the variable name, normalized text input, visible unknown-material path, and labeled output.

NAZARBAYEV UNIVERSITY

Nested dictionary for material properties

```
materials = {
    "steel": {
        "density_kg_m3": 7850,
        "yield_strength_pa": 250e6,
        "elastic_modulus_pa": 200e9
    },
    "aluminum": {
        "density_kg_m3": 2700,
        "yield_strength_pa": 150e6,
        "elastic_modulus_pa": 69e9
    }
}
```

Self-documenting keys

Including units in property keys makes incompatible quantities harder to confuse.

Access a nested property

```
steel_strength_pa = (
    materials["steel"]["yield_strength_pa"]
)
print(f"Steel yield strength = "
      f"{steel_strength_pa / 1e6:.1f} MPa")
```

Read the access path

Select the `"steel"` record, then select its `"yield_strength_pa"` value.

NAZARBAYEV UNIVERSITY

A complete nested lookup with normalized input

```
selected = input("Enter material name: ").strip().lower()

if selected in materials:
    record = materials[selected]
    density = record["density_kg_m3"]
    strength_mpa = record["yield_strength_pa"] / 1e6
    print(f"Material: {selected}")
    print(f"Density: {density} kg/m^3")
    print(f"Yield strength: {strength_mpa:.1f} MPa")
else:
    print("Unknown material")
```

Key consistency

`"aluminum"` and `"Aluminium"` are different keys. Normalization reduces, but does not eliminate, vocabulary mismatches.

NAZARBAYEV UNIVERSITY

Sensor metadata belongs naturally in a dictionary

```
sensor = {
    "id": "T-101",
    "quantity": "temperature",
    "unit": "C",
    "location": "heat exchanger outlet",
    "calibrated": True,
    "warning_limit": 80.0
}
```

Why this works

The fields have different types, and each field's name is more meaningful than a numerical position.

NAZARBAYEV UNIVERSITY

Combine metadata with repeated readings

```
sensor["readings_c"] = [
    72.1, 73.5, 75.0, 81.2, 79.8
]

limit_c = sensor["warning_limit"]
for reading_c in sensor["readings_c"]:
    if reading_c > limit_c:
        print(f"Warning: {reading_c:.1f} C")
```

Structure choice

Dictionary for named fields; list for repeated values of the same kind.

Units

The key `readings_c` and the limit must use the same temperature scale.

NAZARBAYEV UNIVERSITY

Evaluate a list of design dictionaries

```
designs = [
    {"name": "A", "area_mm2": 300, "cost": 12.5},
    {"name": "B", "area_mm2": 450, "cost": 16.0},
    {"name": "C", "area_mm2": 600, "cost": 20.0}
]
force_n = 12000.0
allowable_stress_pa = 80e6

for design in designs:
    area_m2 = design["area_mm2"] / 1_000_000
    stress_pa = force_n / area_m2
    design["stress_mpa"] = stress_pa / 1e6
    design["status"] = (
        "safe" if stress_pa <= allowable_stress_pa else "unsafe")
```

NAZARBAYEV UNIVERSITY

Produce a readable design table

```
print("DESIGN   AREA (mm^2)   STRESS (MPa)   STATUS")
for design in designs:
    print(f"{design['name']:>6s}"
          f"{design['area_mm2']:14.0f}"
          f"{design['stress_mpa']:15.1f}"
          f"{design['status']:>10s}")
```

Separate computation from communication

The dictionaries retain the calculated evidence. The reporting loop decides how to display it.

NAZARBAYEV UNIVERSITY

Choose among list, tuple, and dictionary

Question	List	Tuple	Dictionary
How accessed?	position	position	key
Can change?	yes	no item assignment	yes
Best for	repeated ordered values	fixed short records	named fields and lookup
Example	time readings	(x, y, z) coordinate	sensor metadata
Main risk	index/alias errors	unclear position meaning	key errors

Prediction and debugging activity

```
properties = {
    "steel": {"density": 7850},
    "aluminum": {"density": 2700}
}
material = " Steel "
record = properties.get(material)
print(record["density"])
```

Before running

Predict the failure. Identify the earliest assumption that is violated. Repair the code without adding a fake default density.

Pairs - 5 min

NAZARBAYEV UNIVERSITY

Common dictionary errors

Error pattern	Symptom	Prevention
Missing key	`KeyError`	membership test or meaningful `get()` default
Case/space mismatch	expected record not found	normalize controlled text input
Inconsistent vocabulary	different code sections disagree	define one key schema
Lost units	plausible but wrong result	units in keys, documentation, and output
Unexpected mutation	reference data changes	copy before modification; limit side effects
Wrong nesting path	key access fails at inner level	inspect structure and use clear intermediate records

NAZARBAYEV UNIVERSITY

Dictionary design checklist for engineers

1. Does each key communicate one stable meaning?
2. Are units visible in keys, comments, or documentation?
3. Are required and optional fields identified?
4. What should happen when a key is missing?
5. Are input names normalized without changing technical identifiers incorrectly?
6. Will mutation overwrite authoritative or raw data?
7. Should one record be a dictionary, or should many records be a list of dictionaries?
8. Can another person understand the schema without guessing positions?

NAZARBAYEV UNIVERSITY

Assignment 3 launch: course-aligned focus

Core evidence: trace simple recursion, compare it with iteration, and use dictionaries for named engineering data.

Possible integrated structure

1. recursive numerical and non-numerical functions;
2. engineering records stored in dictionaries;
3. a processed list of records and a clear report;
4. known-value, base-case, and boundary tests.

Authoritative instructions

Use Moodle for the final task, deadline, submission format, permitted resources, collaboration rules, and marking criteria.

NAZARBAYEV UNIVERSITY

Assignment 3 planning template

Planning question	Evidence to prepare
What is the engineering objective?	inputs, outputs, assumptions, units, and decision or report
What recurs?	function name, base cases, smaller input, combination rule
Why does it terminate?	a size measure that decreases on every recursive path
What does each dictionary represent?	key schema, value types, required fields, unit convention
How are missing keys handled?	validation, membership check, or explicit sentinel
How is it verified?	base, near-base, normal, invalid, and known-value tests
What will be submitted?	code, outputs, explanation, tests, and any required AI-use disclosure

NAZARBAYEV UNIVERSITY

Individual understanding and academic responsibility

You remain responsible for every submitted line

Be prepared to explain, trace, reproduce, or modify any recursive function, dictionary operation, output, or engineering interpretation in your submission.

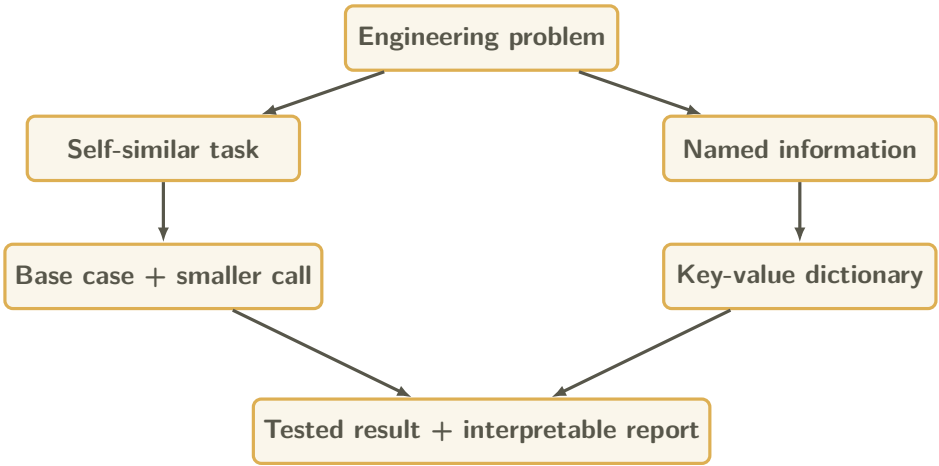
Before seeking assistance

Write the base cases, recursive reduction, dictionary schema, and first tests yourself.

Before submission

Run known cases, inspect every key and unit, restart and run from the beginning, and document permitted assistance as required.

Week 6 concept map



NAZARBAYEV UNIVERSITY

Lecture 18 exit ticket

A pressure sensor record must store its ID, unit, location, warning limit, calibration status, and five readings.

1. Which information belongs in a dictionary?
2. Which information belongs in a list?
3. Write two suitable keys that make units visible.
4. How would you safely retrieve an optional calibration date?
5. State one normal test and one missing-key test.

Submit before leaving

NAZARBAYEV UNIVERSITY

Week 6 quick reference: recursion

Core pattern

```
def recursive_function(problem):
    if base_condition(problem):
        return direct_answer(problem)
    smaller = reduce_problem(problem)
    result = recursive_function(smaller)
    return combine(problem, result)
```

Verify before trusting

1. Identify every base case.
2. State the decreasing size measure.
3. Check every path approaches a base case.
4. Trace returned sub-results.
5. Compare with a loop when uncertain.

Typical defects

Missing or unreachable base case; input does not shrink; missing `return`; valid terminal form omitted.

NAZARBAYEV UNIVERSITY

Week 6 quick reference: dictionaries

Common operations

`d = "key": value`	create
`d[key]`	retrieve; may raise KeyError
`d[key] = value`	add or update
`key in d`	membership test
`d.get(key, default)`	safe retrieval with default
`d.keys()`	keys view
`d.values()`	values view
`d.items()`	key-value pairs
`del d[key]`	delete known key
`d.pop(key, default)`	remove and return

Engineering patterns

Lookup: material name → density.

Record: sensor field name → value.

Nested record: material → property dictionary.

Dataset: list of design dictionaries.

Preserve meaning

Use consistent keys, visible units, explicit missing-data behavior, and copies when authoritative data must remain unchanged.

NAZARBAYEV UNIVERSITY

Suggested reading and source basis

- Nazarbayev University, *ENG 101 - Programming for Engineers: Course Specification Report*, Fall 2026, Session 6.
- Y. Wang, *Programming for Engineers with Python*, working draft, August 2026, Sections 5.13–5.16 and 7.16–7.17.
- J. V. Guttag, *Introduction to Computation and Programming Using Python*, 2nd ed., Sections 4.3 and 5.6.

Preparation habit

Trace recursion on paper, prove that the input becomes smaller, test the base cases first, and write dictionary keys as a small documented data schema.

NAZARBAYEV UNIVERSITY

Reduce complexity without losing meaning

Recursion reduces a problem to smaller instances and rebuilds the answer. Dictionaries connect values to names. In both cases, trustworthy engineering code depends on explicit stopping rules, clear data meaning, units, and verification.

ENG 101 - Programming for Engineers